



UNIVERSIDAD DE LA SABANA

Maestría · Diseño y optimización de bases de datos

Unidad 2 · Actividad Evaluativa

ECOMMIFY

Diseño de arquitectura híbrida PostgreSQL + MongoDB

Dataset Brazilian E-Commerce (Olist)



Integrantes



Santiago Eduardo

Muñoz Castillo

Cód. 0000394453



Edward Augusto

Ramirez Rodriguez

Cód. 0000324316



Wolfran Alirio

Pinzon Murillo

Cód. 0000393439



Henry Julian

Salazar Salcedo

Cód. 0000396117



Agenda

- 01 Resumen ejecutivo y enfoque arquitectónico
- 02 Análisis de requisitos y entidades del dominio
- 03 Diseño conceptual (ER global)
- 04 Diseño lógico PostgreSQL
- 05 Diseño lógico MongoDB
- 06 Decisiones arquitectónicas y CAP
- 07 Plan de implementación y repositorio



Decisiones clave del diseño

- Arquitectura híbrida (políglota): PostgreSQL + MongoDB
- Datos transaccionales (pedidos, pagos, logística) en relacional con ACID
- Datos no estructurados (reseñas, texto libre) en NoSQL para flexibilidad
- Capas: ingestión CSV → data lake → persistencia dual
- Replicación y particionamiento para alta disponibilidad
- Marco escalable y evolutivo para el ecosistema de e-commerce



Arquitectura modular y desacoplada



Cada dominio (clientes, pedidos, pagos, logística, reseñas) se modela como un bounded context independiente, manteniendo integridad relacional donde importa y flexibilidad donde aporta.



Contexto de Ecommify

- Plataforma multivendedor de productos tecnológicos (caso académico)
- Ciclo completo de venta: clientes, catálogo, pedidos, pagos y reseñas
- Dataset Brazilian E-Commerce (Olist) · ~99.000 pedidos · 2016–2018
- Datos transaccionales estructurados + datos analíticos con texto libre
- Decisión: arquitectura híbrida PostgreSQL ↔ MongoDB



Requisitos funcionales y no funcionales

Funcionales

- Gestión de clientes
- Gestión de órdenes y estados
- Procesamiento y tipos de pagos
- Gestión de productos y categorías
- Gestión de vendedores
- Reseñas y calificaciones postventa

No funcionales

- Escalabilidad (volumen alto)
- Integridad referencial
- Rendimiento en consultas frecuentes
- Seguridad de datos sensibles
- Disponibilidad operativa
- Mantenibilidad y evolución



Modelo de negocio Ecommify (Olist)

Núcleo transaccional

- customers · clientes
- orders · pedidos
- order_items · líneas de pedido
- order_payments · pagos
- sellers · vendedores

Catálogo y analítico

- products · catálogo de productos
- product_category · categorías
- order_reviews · reseñas
- geolocation · ubicación postal



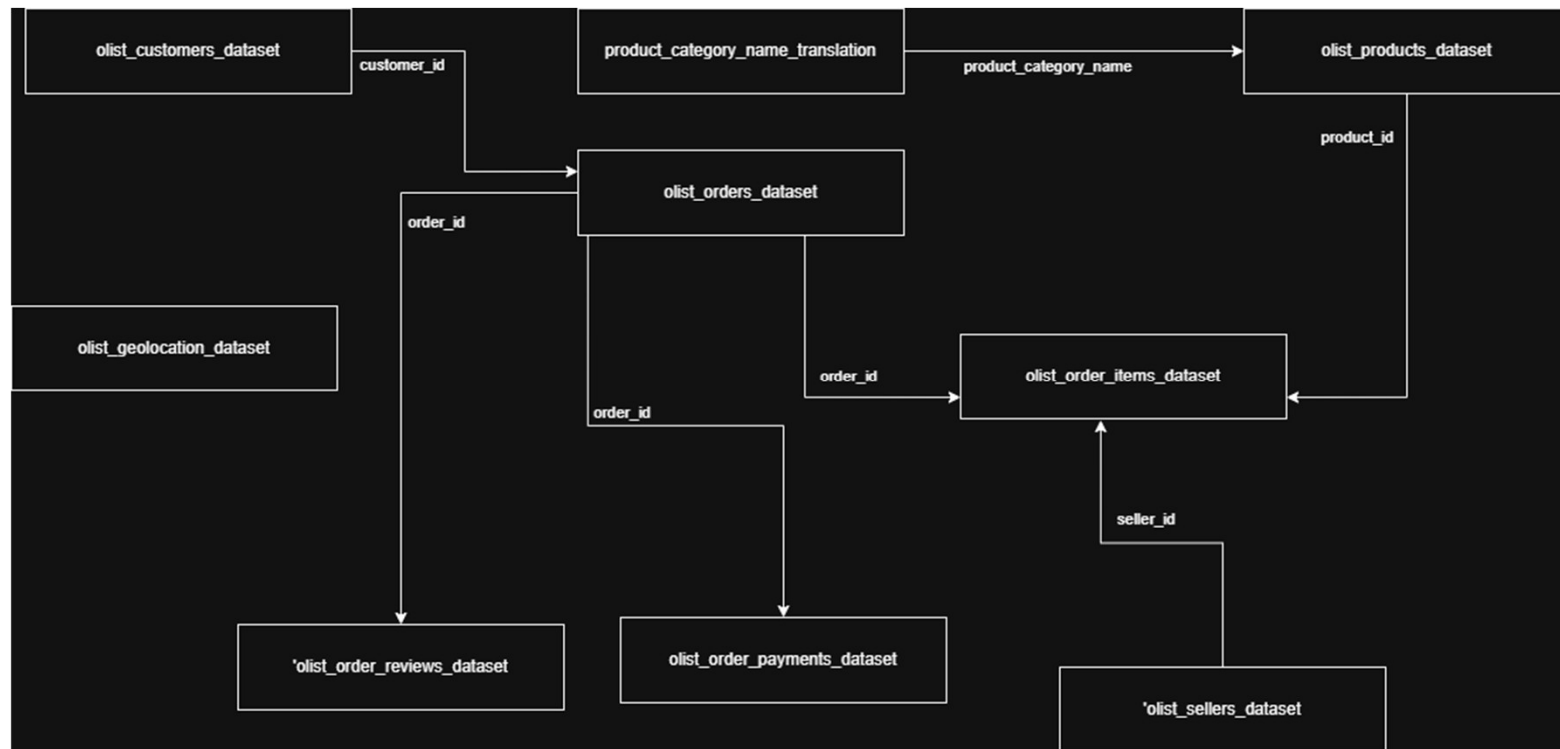
Tabla de volúmenes de datos

Entidad	Crec. diario	Crec. anual	Tamaño/reg.	Volumen anual
customers	50	18 250	2 KB	36.5 MB
orders	500	182 500	3 KB	547.5 MB
payments	500	182 500	2 KB	365 MB
products	10	3 650	4 KB	16.7 MB
sellers	2	730	2 KB	1.46 MB

Proyección estimada para el ejercicio académico. Base inicial tomada del dataset Olist (2016–2018).



Diagrama Entidad-Relación global (Olist)

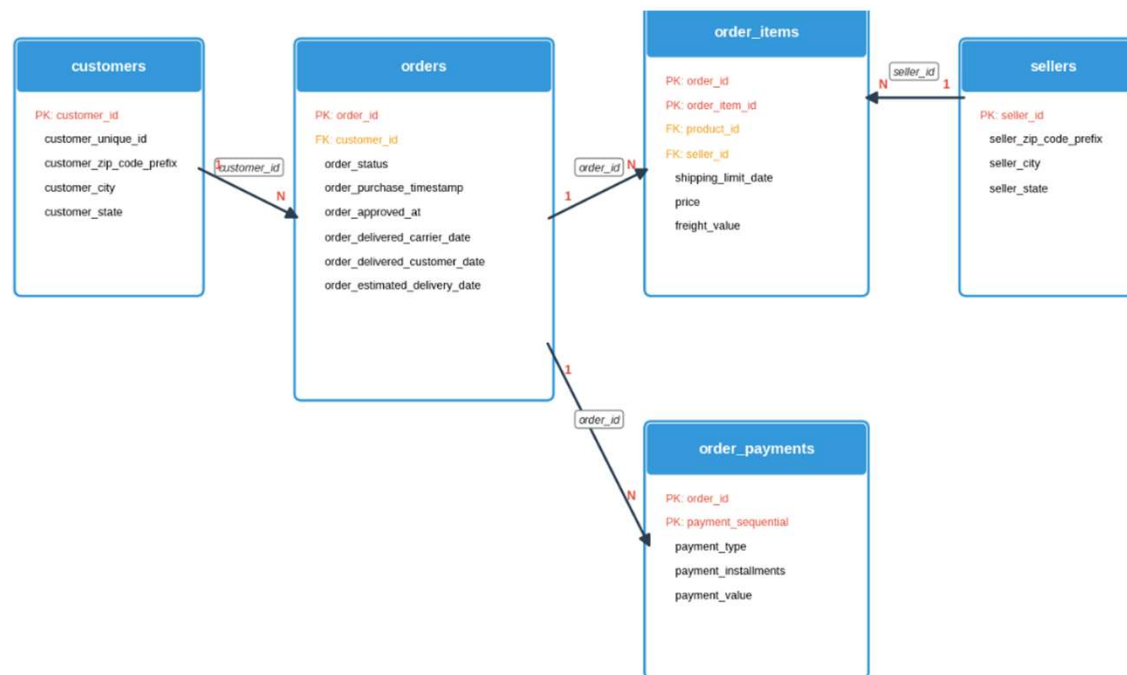


Tablas y claves de unión del dataset Brazilian E-Commerce (Olist).



Modelo Normalizado (3FN)

Diagrama Entidad Relación: Modelo Normalizado (3FN)



customers · orders · order_items · order_payments · sellers (particionamiento HASH en order_items).



Reglas que rigen el diseño

- Segregación por tipo: transaccional (PG) vs no estructurado (MongoDB)
- Integridad referencial mediante claves foráneas
- Claves primarias únicas (simples o compuestas) y NOT NULL
- Atomicidad 1FN: fechas y timestamps con tipo correcto
- Gestión de duplicados por identificadores clave (drop_duplicates)
- Validaciones de dominio (CHECK en montos y cuotas)



Aplicación de tipos avanzados

- Decisión: no usar JSONB / Arrays en PostgreSQL
- Motivo: arquitectura híbrida; la flexibilidad vive en MongoDB
- Evita complejidad en consultas y mantenimiento del modelo
- Conserva claridad arquitectónica y validación de datos
- PostgreSQL queda dedicado a estructuras 3FN con ACID



Colecciones principales

products

Catálogo de productos con atributos variables y dimensiones

order_reviews

Valoraciones y comentarios en texto libre

product_categories

Jerarquía y traducción de categorías

geolocation

Códigos postales y coordenadas (GeoJSON)



Colección products (JSON)

JSON

```
{
  "product_id": "ABC123",
  "product_category": {
    "name": "beleza_saude",
    "name_english": "health_beauty"
  },
  "product_name_length": 25,
  "product_description_length": 120,
  "product_photos_qty": 3,
  "dimensions": {
    "weight_g": 1500,
    "length_cm": 30,
    "height_cm": 10,
    "width_cm": 20
  },
  "metadata": {
    "created_at": "2026-05-23T08:06:00Z",
    "updated_at": "2026-05-23T08:06:00Z"
  }
}
```



Colecciones order_reviews y geolocation

order_reviews

```
{
  "review_id": "R12345",
  "order_id": "098765",
  "customer_id": "C54321",
  "review_score": 4,
  "review_comment_title": "Entrega rápida",
  "review_comment_message": "...",
  "review_creation_date": "...",
  "review_answer_timestamp": "..."
}
```

geolocation

```
{
  "geolocation_zip_code_prefix": "01001",
  "location": {
    "type": "Point",
    "coordinates": [-46.6333, -23.5505]
  },
  "geolocation_city": "São Paulo",
  "geolocation_state": "SP",
  "metadata": { ... }
}
```




Patrones MongoDB aplicados por colección

Colección	Patrón	Razón
products	Embedding	Embebe product_category; optimiza lecturas eliminando joins y garantiza atomicidad.
order_reviews	Referencing	IDs hacia orders y customers: datos voluminosos y actualizables por separado.
geolocation	Computed	Resultados precalculados; evita operaciones espaciales repetitivas.

Cada patrón se elige según el patrón de acceso y el tamaño del documento.



Criterios de decisión

- Integridad y transacciones (ACID) → PostgreSQL
- Relaciones complejas y muchas FK → PostgreSQL
- Esquema variable o subdocumentos → MongoDB
- Lectura analítica y filtros flexibles → MongoDB
- CAP: transaccional prioriza Consistencia · analítico prioriza Disponibilidad



Matriz de decisión por entidad

Entidad	Tabla fija	ACID	Esquema variable	Relación compleja	Motor
customers	Sí	Sí	No	Sí (1:N)	PostgreSQL
orders	Sí	Sí	No	Sí (núcleo)	PostgreSQL
order_items	Sí	Sí	No	Sí (pedido+prod+vend)	PostgreSQL
order_payments	Sí	Sí	No	Media	PostgreSQL
sellers	Sí	Sí	No	Media	PostgreSQL
products	No	No	Sí	Baja (por ID)	MongoDB
order_reviews	No	No	Sí	Baja (por order_id)	MongoDB
product_category	Sí	No	Parcial	Baja	MongoDB
geolocation	Sí	No	Parcial	Baja	MongoDB



Teorema CAP • Trade-offs y mitigación

Análisis CAP

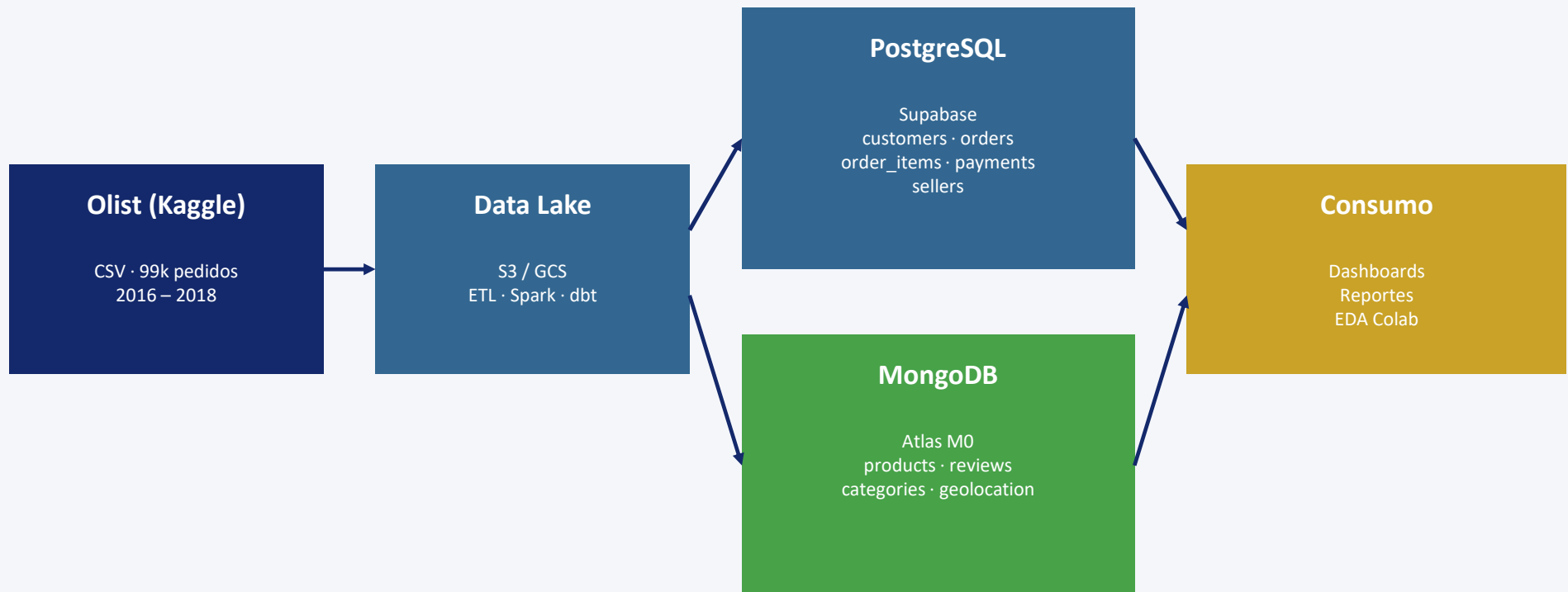
- Módulo transaccional → Consistencia (C)
- Módulo analítico/documental → Disponibilidad (A)
- PG garantiza ACID en pedidos y pagos
- MongoDB favorece lectura flexible

Trade-offs

- Mayor complejidad de integración entre motores
- Replicación y particionamiento para resiliencia
- Consistencia fuerte donde importa el dinero
- Flexibilidad donde importa el análisis



Flujo de datos Ecommify





Hoja de ruta técnica

1

Crear esquema PostgreSQL en Supabase

Ejecutar DDL · particiones · constraints

2

Configurar MongoDB Atlas (M0)

Crear colecciones, índices y validaciones

3

Cargar datos Olist

Seed SQL en PostgreSQL · scripts en MongoDB

4

EDA en Google Colab

Análisis exploratorio sobre los datos cargados

5

Consultas de validación

queries.sql · agregaciones MongoDB

6

Repositorio GitHub

docs / postgresql / mongodb / notebooks



Repositorio GitHub

github.com/earamirez190/Ecommify_Database_Design

```
Ecommify_Database_Design/  
├── README.md  
├── docs/  
│   └── Documento_Técnico_Diseño.docx  
├── postgresql/  
│   ├── schema/ (Modelo Fisico.sql · ER ModeloConceptual.xlsx)  
│   ├── seed_data/(customers_rows.sql · order_items_rows.sql)  
│   └── queries/ (queries.sql)  
├── mongodb/  
│   └── schema/ (Diseño preliminar NoSQL MongoDB.docx · Matriz_DecisionesARQ.xlsx)  
└── notebooks/  
    ├── Data_Exploration_Analysis.ipynb  
    └── Normalizacion_Ecommify_Olist.ipynb
```

Estructura conforme a la guía de la Unidad 2.



Cierre del entregable

- Arquitectura híbrida coherente con la naturaleza de los datos de Olist
- PostgreSQL: integridad y consistencia en el núcleo de ventas
- MongoDB: flexibilidad y análisis en catálogo, reseñas y geo
- Patrones aplicados (embedding, referencing, computed) justificados
- Trade-offs CAP aceptados y mitigados con replicación y particionamiento
- Diseño listo para implementación en Supabase + MongoDB Atlas



Gracias